KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

Personal Health Record Monitoring System

Submitted by

R.RITHICK

25BIT048

Under the Guidance of

DHILEEPAN A

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: 2025 – 2026

OCTOBER 2026

Semester: I

DECLARATION

I hereby declare that the project entitled “Personal Health Record Monitoring System” submitted in partial fulfillment of the requirements for the course Java programming language is a record of my original work carried out under the guidance of Dhileepan A, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

_____

Prasanya S

CERTIFICATE

This is to certify that the project entitled “Personal Health Record Monitoring System” is a bonafide work carried out by R.Rithick, IT, INFORMATION TECHNOLOGY, KG College of Arts & Science, in partial fulfillment of the requirements for the course Java programming language.

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place: Coimbatore

Date:

Submitted for the Viva-Voce Examination held on

Guide:
(Technical Trainer, KGCAS)

Head of the Department
(Technical Head, KGCAS)

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	
1.1	Objective	
1.2	Overview	
1.3	Features	
2	**Program Analysis**	
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scopes and Limitations	
3	**System Design**	
3.1	Modules	
4	**Methodology**	
4.1	Problem Analysis	
4.2	System Design	
4.3	Program Development	
4.4	Testing and Validation	
4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	**Source Code**	
6	**Input and Output Screens**	
7	**Discussion and Conclusion**	
7.1	Discussion	
7.2	Conclusion	
8	**References**	

ABSTRACT

The Personal Health Record Monitoring System is a Java-based application developed to manage and monitor health records in an organized and secure manner. In many organizations, health record is recorded manually in registers, which can lead to errors, data loss, and difficulty in tracking health details.

This system provides a computerized solution to log patient entries, store their details, and monitor access efficiently. The application allows users to add health information such as name, health notes, record date and time, next checkup date, and contact details. It also enables searching, updating, and deleting health records.

The system uses file handling to ensure permanent storage of data. It is developed using Core Java concepts such as classes, objects, ArrayList, loops, and conditional statements.

The main objective of this project is to enhance security, reduce manual effort, and provide a reliable system for tracking health measurements. This project demonstrates the practical implementation of Java in real-world security applications.

1. INTRODUCTION

In today's world, security and monitoring of patients is very important in places like offices, colleges, apartments, and industries. Traditionally, health details are recorded manually in registers, which can be inefficient and prone to errors.

Manual systems face issues such as:

- Data misplacement
- Time-consuming searches
- Lack of security
- No proper tracking

To overcome these issues, a computerized Personal Health Record Monitoring System is developed.

This system allows users to:

- Record health details
- Track entry and exit
- Maintain secure records
- Retrieve information quickly

The system is developed using Core Java and can be extended to GUI or database integration.

1.1 Objective

The main objectives of this project are:

- To develop a Java-based health record management system
- To record health record and exit details
- To improve health monitoring
- To reduce manual errors
- To provide quick search and retrieval
- To store data permanently using file handling
- To demonstrate Core Java concepts

1.2 Overview

The system is a menu-driven application where users can:

- Add health details
- View all health records
- Search health information
- Update details
- Delete records

Each health record contains:

- Patient ID
- Name
- Contact number
- Purpose of visit
- Entry time
- Exit time

The system ensures efficient and secure patient tracking.

1.3 Features

Personal Health Record Monitoring

- Entry and Next Checkup Date Tracking
- Add Patient Details
- View All Patients
- Search Patient
- Update Patient Information
- Delete Patient Records
- File Handling for storage
- User-friendly interface

2. PROGRAM ANALYSIS

Program analysis is a crucial phase in software development where the problem is studied in detail and system requirements are clearly identified. This phase helps in understanding the existing system, identifying its drawbacks, and designing an efficient computerized solution.

In the case of the Personal Health Record Monitoring System, the analysis focuses on improving patient tracking, enhancing security, and reducing manual effort through automation.

2.1 Problem Definition

In many organizations such as offices, colleges, hospitals, and residential apartments, health details are recorded manually in registers. While this method is simple, it has several disadvantages that affect both efficiency and security.
The major problems in the existing system include:

1. Manual Record Maintenance

Health details are written in physical registers, which can be easily damaged, lost, or misplaced. Managing large volumes of records becomes difficult over time.

2. Time-Consuming Process

Entering health details manually takes time, especially when there are many patients. Searching for a particular health record in registers is also very slow.

3. Lack of Security

Manual systems do not provide proper monitoring. It is difficult to track suspicious patients or verify entry and next checkup dates accurately.

4. Data Inaccuracy

Human errors such as incorrect entries, incomplete details, or illegible handwriting can lead to unreliable data.

5. No Proper Tracking System

There is no efficient way to track:

- Entry time
- Exit time
- Purpose of visit

This reduces the effectiveness of health monitoring.

6. No Data Backup

If registers are lost or damaged, all data is permanently lost since there is no backup.

Need for the Proposed System;

To overcome these issues, there is a need for an automated system that:

- Stores patient data securely
- Allows quick access and retrieval
- Tracks health information efficiently
- Reduces manual work and errors

The Personal Health Record Monitoring System is designed to solve these problems effectively.

2.2 Project Overview

The Personal Health Record Monitoring System is a Java-based application that provides a simple and efficient way to manage health records digitally.
The system is designed as a menu-driven console application, where users can perform various operations related to health record management.

Working of the System

1. When the program starts, a menu is displayed with different options.
2. The user selects an operation based on their requirement.
3. The system performs the selected operation and displays the result.

Main Functionalities

1. Add Patient Record

The system allows the user to enter health details such as:

- Name
- Contact number
- Purpose of visit
- Entry time
- Exit time

- Each patient is assigned a unique ID.

2. View Patient Records

- Displays all stored health details in a structured format.
- Helps users monitor patient activity.

3. Search Patient

- Allows searching health records using name or ID.
- Provides quick access to required information.

4. Update Patient Details

- Enables modification of health information.
- Useful when details need correction.

5. Delete Patient Record

- Removes unwanted or outdated records.
- Keeps the system clean and organized.

6. File Handling

- Stores all patient data in files.
- Ensures data is not lost when the program is closed.

Advantages of the Proposed System

- Faster data entry and retrieval
- Improved health monitoring
- Accurate record maintenance
- Reduced manual effort
- Reliable data storage

2.3 Scopes and Limitations

Scope of the System;

The Personal Health Record Monitoring System has a wide range of applications in different areas:

1. Offices and Corporate Buildings

- To monitor employee patients
- Maintain health logs

2. Educational Institutions

- Track patients entering campuses
- Maintain safety of students

3. Hospitals

- Manage patient patients
- Control entry access

4. Residential Apartments

- Monitor guest entries
- Improve resident security

5. Small Organizations

- Replace manual registers with digital systems

Limitations of the System

Despite its advantages, the system has some limitations:

1. Console-Based Interface

- The system does not have a graphical interface, which may not be very user-friendly.

2. Limited Security Features

- No authentication or login system in the basic version.

3. No Real-Time Monitoring

- Does not provide live alerts for suspicious activities.

4. File-Based Storage

- Data is stored in files instead of a database, which may not be suitable for very large systems.

5. Single User System

- Designed for basic usage, not suitable for multi-user environments.

3. SYSTEM DESIGN

System design is a crucial phase in software development where the overall structure of the system is planned before implementation. It defines how the system will work, how different components interact, and how data flows within the system.

In the Personal Health Record Monitoring System, the design is focused on creating a simple, efficient, and secure way to manage health records using a menu-driven Java application.

The system is divided into multiple modules, each responsible for a specific function. This modular design makes the system easy to understand, develop, and maintain.

3.1 Modules

The system is divided into the following major modules:

1. Health Record Module

This module is responsible for adding new health records into the system.

Functions:

- Accept health details from the user
- Validate input data
- Assign a unique Patient ID
- Store the record in memory and file

Input Details:

- Patient Name
- Contact Number
- Health Notes
- Record Date and Time
- Next Checkup Date

2. View Patient Module

This module displays all stored health records in a structured format.

Functions:

- Retrieve all records from storage

- Display them clearly

Purpose:

- Helps security staff monitor all patient entries
- Provides a complete overview

3. Search Patient Module

This module helps in locating a specific health record quickly.

Functions:

- Accept search input (Name or ID)
- Compare with stored records
- Display matching result

Purpose:

- Saves time
- Enables quick verification

4. Update Patient Module

This module allows modification of existing health records.

Functions:

- Select patient using ID
- Update required fields
- Save updated data

Purpose:

- Correct errors
- Keep records accurate

5. Delete Patient Module

This module removes unwanted or outdated health records.

Functions:

- Select record using ID
- Delete the record
- Update storage

Purpose:

- Maintain clean database
- Remove unnecessary data

6. File Handling Module

This module ensures permanent storage of patient data.

Functions:

- Save records into a file
- Load records when program starts
- Prevent data loss

Purpose:

- Data persistence
- Reliability

Techniques Used:

- FileWriter
- BufferedReader
- Serialization (optional)

7. Menu / Control Module

This module controls the overall flow of the system.

Functions:

- Display menu options

- Accept user choice
- Call appropriate module

8. Exit Module

This module safely terminates the program.

Functions:

- Save all data
- Exit application

4. METHODOLOGY

Methodology describes the systematic approach followed to develop the project. It explains the step-by-step process used to analyze the problem, design the system, implement the solution, and test its functionality.

In this project, a structured and sequential development process was followed to ensure that the Personal Health Record Monitoring System meets user requirements and functions efficiently.

The methodology consists of the following phases:

1. Problem Analysis
2. System Design
3. Program Development
4. Testing and Validation
5. Result and Documentation

4.1. Problem Analysis

This is the first and most important phase of the project development.

Objective of this Phase:

- To understand the existing system
- To identify problems and limitations
- To define system requirements

Activities Performed:

- Studied manual patient register systems
- Identified issues such as:
    - Data loss
    - Time-consuming search
    - Lack of security
    - Poor record management

Conclusion of Analysis:

It was concluded that an automated system is required to:

- Maintain health records efficiently
- Provide quick search functionality
- Ensure secure and reliable data storage

This phase helped in defining the overall goals of the project.

4.2. System Design

In this phase, the structure of the system is planned before coding begins.

Objective:

- To design how the system will work
- To divide the system into modules

Design Approach:

The system is designed as a **menu-driven console application**.

Modules Identified:

- Health Record Module
- View Patient Module
- Search Module
- Update Module
- Delete Module
- File Handling Module
- Exit Module

Design Considerations:

- Simplicity and ease of use
- Efficient data handling
- Modular structure for easy maintenance

This phase ensures that the system is well-organized and easy to implement.

4.3. Program Development

This phase involves actual coding and implementation of the system using Java.

Technologies Used:

- Core Java
- File Handling
- ArrayList
- Scanner Class

Steps Involved:

1. **Class Creation**
    - Created a Patient class to store health details
    - Created a main class for system operations
2. **Menu Implementation**
    - Developed a menu-driven interface
    - Used loops and switch-case statements
3. **Functional Implementation**
    - Add patient
    - View patients
    - Search patient
    - Update patient
    - Delete patient
4. **Data Storage**
    - Implemented file handling for saving records
5. **User Input Handling**
    - Used Scanner class for input

Concepts Used:

- Classes and Objects
- Methods
- Loops (while, for)
- Conditional statements (if, switch)
- Collections (ArrayList)

This phase converts the system design into a working application.

4.4. Testing and Validation

Testing ensures that the system works correctly and produces accurate results.

Objective:

- To identify and fix errors
- To verify system functionality

Types of Testing Performed:

1. Functional Testing

- Checked whether each module works correctly

2. Input Testing

- Tested valid and invalid inputs

3. Output Testing

- Verified correctness of results

Test Case	Input	Expected Output
Add Patient	Valid details	Patient added successfully
View Patients	-	Display all records
Search Patient	Existing name	Record found
Update Patient	Valid ID	Data updated
Delete Patient	Valid ID	Record deleted
Invalid Input	Wrong ID	Error message

This phase ensures that the program runs without errors and produces accurate results.

4.5. Result and Documentation

This is the final phase of the project.

Objective:

- To present the working system
- To document the entire project

Results Achieved:

The system successfully:

- Stores health details
- Displays all records

- Searches health information quickly
- Updates and deletes records
- Maintains data using file handling

4.6. Algorithm

- Start the program
- Initialize patient list
- Display menu
- Read user choice
- If choice = Add → enter and store health details
- If choice = View → display all patients
- If choice = Search → find and display patient
- If choice = Update → modify health details
- If choice = Delete → remove health record
- If choice = Exit → stop program
- Repeat steps until Exit
- End program

4.7. Flowchart

Start

↓

Display Menu

↓

Enter Choice

↓

Check Choice

↓

Add , View , Search , Update ,Delete

↓

Perform Operation

↓

Display Result

↓

Exit?

↓ ↓

No Yes

↓ ↓

Menu End

5. SOURCE CODE

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class PatientEntry extends JFrame implements ActionListener {

    JLabel nameLabel, purposeLabel, timeLabel;
    JTextField nameField, purposeField;
    JButton submitButton;

    public PatientEntry() {

        setTitle("Personal Health Record Monitoring System");
        setSize(400,300);
        setLayout(new FlowLayout());

        nameLabel = new JLabel("Patient Name:");
        nameField = new JTextField(20);

        purposeLabel = new JLabel("Purpose:");
        purposeField = new JTextField(20);

        submitButton = new JButton("Submit");

        timeLabel = new JLabel("");

        add(nameLabel);
        add(nameField);
        add(purposeLabel);
        add(purposeField);
        add(submitButton);
        add(timeLabel);

        submitButton.addActionListener(this);

        setVisible(true);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }

    public void actionPerformed(ActionEvent e) {

        String name = nameField.getText();
        String purpose = purposeField.getText();

        String time = java.time.LocalTime.now().toString();

        timeLabel.setText("Entry Recorded at: " + time);

        JOptionPane.showMessageDialog(this,
        "Patient Logged\nName: " + name +
        "\nPurpose: " + purpose +
```

```
        "\nTime: " + time);
    }

    public static void main(String[] args) {
        new PatientEntry();
    }
}
```

6. INPUT AND OUTPUT SCREENS

7. DISCUSSION & CONCLUSION

7.1 Discussion

The Personal Health Record Monitoring System was developed to provide an efficient and reliable solution for managing health records in organizations. The system replaces traditional manual record-keeping methods with a computerized approach, improving both efficiency and security.

During the development of this project, various **Core Java concepts** such as classes, objects, methods, loops, conditional statements, ArrayList, and file handling were successfully implemented. These concepts helped in building a structured and functional application.

The system allows users to perform essential operations such as:

- Adding health details
- Viewing all health records
- Searching for specific patients
- Updating existing records
- Deleting unwanted records

Each module was designed and tested individually to ensure proper functionality. The integration of all modules resulted in a smooth and user-friendly system.

7.2 Conclusion

The Personal Health Record Monitoring System successfully achieves its objective of providing a simple, efficient, and secure method for managing health records.

The system demonstrates how Java programming can be used to develop real-world applications that solve practical problems. It reduces the dependency on manual systems and improves the overall process of health record management.

Key Achievements

- Developed a functional health record management system
- Implemented all basic CRUD operations (Create, Read, Update, Delete)
- Ensured permanent data storage using file handling
- Created a user-friendly and efficient application

8. REFERENCES

The following resources were used during the development of this project:

Books

• Herbert Schildt – *Java: The Complete Reference*

Websites

• https://www.oracle.com/java
• https://www.geeksforgeeks.org
• https://www.w3schools.com
• https://www.javatpoint.com
•